A3: Databases - Exam Lab

This exam practice page is designed for the A3: Databases unit of the new IB Computer Science syllabus (first assessment 2027).

Syllabus Links:

A3.1 Database fundamentals

A3.2 Database design

A3.3 Database programming

A3.4 Alternative databases and data warehouses (HL only)

The Exam Scenario

Scenario: "GlobalShop" E-commerce Platform. GlobalShop is a fast-growing international online retailer. Currently, they use a central **Relational Database Management System (RDBMS)** to manage their inventory, customer data, and order processing. The database contains tables such as `CUSTOMERS`, `ORDERS`, `PRODUCTS`, and `ORDER_LINE_ITEMS`. Customers have unique IDs and unique email addresses. An order can contain multiple different products in varying quantities. A product has a unique SKU (Stock Keeping Unit). To support their global growth, GlobalShop is considering moving from a centralised database to a **distributed database** architecture.

Core Questions (HL and SL)

Give these questions a try! If you're stuck, check out the 'Strategy' section to help you get started. Once you're finished, use the Mark Scheme and Worked Example to review your progress. If these feel a bit tough right now, head back to the eBook pages to sharpen your understanding first.

Question 1: Data Modeling & Integrity (SL & HL)

(a) Based on the scenario, **construct** an Entity-Relationship Diagram (ERD) showing the relationship between `ORDERS`, `PRODUCTS`, and `ORDER_LINE_ITEMS`. Specify the **cardinality** of the relationships. **[3 marks]**

(b) **Identify** appropriate **Primary Keys (PK)** for the `ORDERS` and `ORDER_LINE_ITEMS` tables. **Suggest** a **Composite Primary Key** if necessary and explain your choice. **[3 marks]**

Level 1: Strategy

Command Terms: "**Construct**" (present in visual/diagram form); "**Identify**" (specific answer); "**Suggest**" (give an opinion/solution for a scenario).

The Logic: A product can appear in many orders; an order can contain many products. `ORDER_LINE_ITEMS` resolves this many-to-many relationship. A PK must uniquely identify a row. If one field isn't enough, you need a composite key.

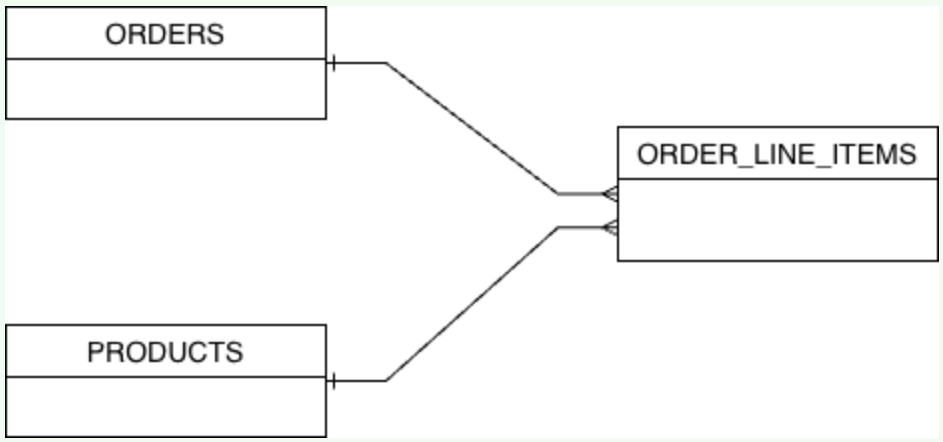
Level 2: Mark Scheme

(a) Award **[1]** for three correct entities (`ORDERS`, `PRODUCTS`, `ORDER_LINE_ITEMS`); **[1]** for lines showing relationships; **[1]** for correct cardinality (e.g., *ORDERS* 1---M *ORDER_LINE_ITEMS* M---1 *PRODUCTS*).

(b) Award **[1]** for suitable PK for `ORDERS` (e.g., `OrderID`). Award **[1]** for suggesting a composite key for `ORDER_LINE_ITEMS` (e.g., `OrderID` + `SKU`). Award **[1]** for explanation (neither field alone is unique within `ORDER_LINE_ITEMS`, but the combination is).

Level 3: Worked Example

(a)



(b) A suitable Primary Key for `ORDERS` would be a unique `OrderID`. For `ORDER_LINE_ITEMS`, neither `OrderID` nor `SKU` alone is sufficient, as multiple rows will share the same `OrderID` (different items in the same order) and multiple rows will share the same `SKU` (same product in different orders).

Therefore, a composite primary key consisting of (`OrderID` + `SKU`) is appropriate to uniquely identify each row in this table.

Question 2: Normalisation (SL & HL)

GlobalShop initially used a flat file to track product information in its regional warehouses, resulting in redundant data:

Product_Name	SKU (PK)	Warehouse_Location	Warehouse_Address	Qty_in_Stock
UltraTab 10	SKU-001	London_Central	1 Main St, London	500
Case 10	SKU-A05	London_Central	1 Main St, London	1200
UltraTab 10	SKU-001	NYC_East	5 River Rd, NYC	300

(a) The table above contains **redundant data**. **Describe** one operational problem that could occur because of this redundancy if the London warehouse changes its address. **[2 marks]**

(b) **Construct** the resulting tables after **normalising** the table above to Third Normal Form (3NF). You must explicitly show the **Primary Keys (PK)** and **Foreign Keys (FK)**. **[6 marks]**

Level 1: Strategy

The Logic: (a) is an update anomaly. If we change the address in one row but forget another, the data is inconsistent. (b) requires moving from unnormalised to 3NF. Separate dependencies: Warehouse data depends on Location, not SKU.

Level 2: Mark Scheme

(a) Award **[1]** for identifying an "Update Anomaly" / Inconsistency. Award **[1]** for explaining in context (if one record is updated but not others, the database holds two different addresses for the same location).

(b) Award **[1]** for identifying correct separate entities. Award **[2]** for correct implementation of 1NF/2NF. Award **[2]** for correct implementation of 3NF (transitive dependency of address removed). Award **[1]** for correctly identifying all PKs and FKs.

Level 3: Worked Example

(a) An **update anomaly** would occur. If the address for `London_Central` changes, the system administrator must update every single product row that lists that warehouse. If they miss one row

(e.g., the row for "Case 10"), the database will contain contradictory or inconsistent information about the warehouse's address.

(b) 3NF Tables:

WAREHOUSES

Location (PK)	Address
London_Central	1 Main St, London
NYC_East	5 River Rd, NYC

PRODUCTS

SKU (PK)	Product_Name
SKU-001	UltraTab 10
SKU-A05	Case 10

INVENTORY

SKU (PK, FK)	Location (PK, FK)	Qty_in_Stock
SKU-001	London_Central	500
SKU-A05	London_Central	1200
SKU-001	NYC_East	300

HL Only: Advanced Networking

Give these questions a try! If you're stuck, check out the 'Strategy' section to help you get started. Once you're finished, use the Mark Scheme and Worked Example to review your progress. If these feel a bit tough right now, head back to the eBook pages to sharpen your understanding first.

Question 3: Transactions & ACID (HL Only)

During GlobalShop’s "Black Friday" sale, two different users try to purchase the exact same "limited stock" item at the same microsecond. The stock level is currently 1. User A initiates a transaction (T1), and User B initiates a transaction (T2).

(a) **Define** the term **Database Transaction**. **[1 mark]**

(b) The system must adhere to **ACID properties**. **Evaluate** why the property of **Consistency** is essential in this scenario. **[3 marks]**

(c) **Explain** how a **concurrency control strategy** is used by the DBMS to ensure the property of **Isolation** between T1 and T2. **[3 marks]**

Level 1: Strategy

Command Terms: "**Define**" (precise meaning); "**Evaluate**" (weigh strengths/weaknesses/implications); "**Explain**" (give reasons or causes).

The Logic: ACID ensures reliable transactions. (b) Consistency is about adhering to rules (inventory cannot be <0). (c) Isolation is about preventing interference (locking).

Level 2: Mark Scheme

(a) Award **[1]** for precise definition (a single logical unit of work that contains one or more database operations).

(b) Award **[1]** for stating that Consistency ensures the database moves from one valid state to another. Award **[1]** for contextualising a valid state (inventory cannot be negative). Award **[1]** for explaining the failure implication (violating the rule that you cannot sell stock you do not have). Award **[1]** for explaining the effect on the primary transaction (e.g., T1 claims exclusive access).

(c) Award **[1]** for identifying a valid concurrency control method (e.g., locking). Award **[1]** for explaining the effect on the primary transaction (e.g., T1 claims exclusive access). Award **[1]** for explaining the effect on the second transaction (e.g., T2 is forced to wait until T1 commits/aborts).

Level 3: Worked Example

(a) A database transaction is a single, logical unit of work composed of one or more sequential operations (like read and write), which must be executed in its entirety or not at all.

(b) Consistency is crucial because it ensures the database always adheres to predefined **data integrity rules**. In this "limited stock" scenario, a rule might be "Stock cannot be negative." Without consistency, both User A and B might successfully decrement the stock from 1 to 0, or even -1 if they read the value simultaneously. Consistency guarantees that after both transactions attempt to run, the database's final state is valid—in this case, only one transaction succeeds, the stock becomes 0, and the other is aborted because selling another would violate the rule.

(c) To ensure **Isolation**, the DBMS implements **Concurrency Control** using **locks**. When User A's transaction (T1) begins, the DBMS places an exclusive lock on the item's data record. When User B's transaction (T2) attempts to read the same record a microsecond later, the DBMS detects the lock and forces T2 to wait. Only once T1 has either successfully committed (selling the item) or aborted (releasing the lock) can T2 proceed, ensuring that User B cannot read or interfere with the intermediate state of the inventory being handled by User A.

Question 4: Database Architectures (HL Only)

GlobalShop is considering upgrading its current centralised database to a **distributed database** architecture to support regional operations.

(a) GlobalShop wants to implement **horizontal fragmentation**. **Describe** what is meant by horizontal fragmentation in this context. **[2 marks]**

(b) **Discuss** the strengths and limitations of GlobalShop implementing a **fully replicated** distributed database architecture compared to their current centralised system. **[6 marks]**

Level 1: Strategy

Command Terms: "**Describe**" (detailed account); "**Discuss**" (balanced review, looking at different aspects/viewpoints).

The Logic: Horizontal fragmentation splits data by rows (e.g., all UK customers go to the UK node). Replicated is having the *full* DB on *every* node.

Level 2: Mark Scheme

(a) Award **[1]** for stating the table is divided by rows. Award **[1]** for context (e.g., splitting based on regional criteria like "all European customers stored on European node").

(b) Award **[1-2]** for describing the comparison (Fully replicated vs. Centralised). Award **[1-2]** for identifying strengths of fully replicated (e.g., extremely high availability/fault tolerance; super-fast local read access). Award **[1-2]** for identifying limitations of fully replicated (e.g., extreme update complexity to maintain **ACID Consistency** across all nodes; high storage costs). Award **[1]** for a conclusion regarding scalability/cost trade-offs.

Level 3: Worked Example

(a) Horizontal fragmentation involves dividing a database table into segments based on subsets of rows. In GlobalShop's context, the `CUSTOMERS` table could be fragmented such that all customer records with a `Country` of "UK" are stored exclusively on a UK server node, while records with a `Country` of "USA" are stored on a USA server node.

(b) A **fully replicated** architecture means every regional node holds a complete copy of the entire database. A significant **strength** of this setup compared to a centralised system is extremely **high availability and fault tolerance**. If the main server fails, regional nodes continue operating; furthermore, local read requests are extremely fast as the data is already on-site. The critical **limitation**, however, is **update complexity**. Every single inventory update or new customer record must be synchronised across *all* nodes globally to maintain consistency. In a high-volume retailer, this "concurrency control" overhead can cripple write performance and significantly increase storage costs and network bandwidth required at every regional centre, making it difficult to scale globally compared to a more segmented distributed architecture.

How confident do you feel with this topic?

Exam Ready: I answered most questions correctly without looking at the eBook. I can explain these concepts to someone else and understand the "why" behind the mark scheme.

Move on to the next topic.

Practitioner: I understood the general concepts, but I missed a few marks on specific terminology or complex applications. I needed the "Strategy" guide for the harder questions.

Annotate your answers with the Mark Scheme and revisit the eBook sections where you lost marks.You can also use the bookmarking and notes feature on the the eBook pages to highlight certain areas.

Novice: I struggled to get started on the questions. I found it difficult to apply the theory to the practice scenarios.

Go back to the eBook and Worked Examples before attempting these questions again. Perhaps use some of the Quizlets to practice the key terms and review the command terms.